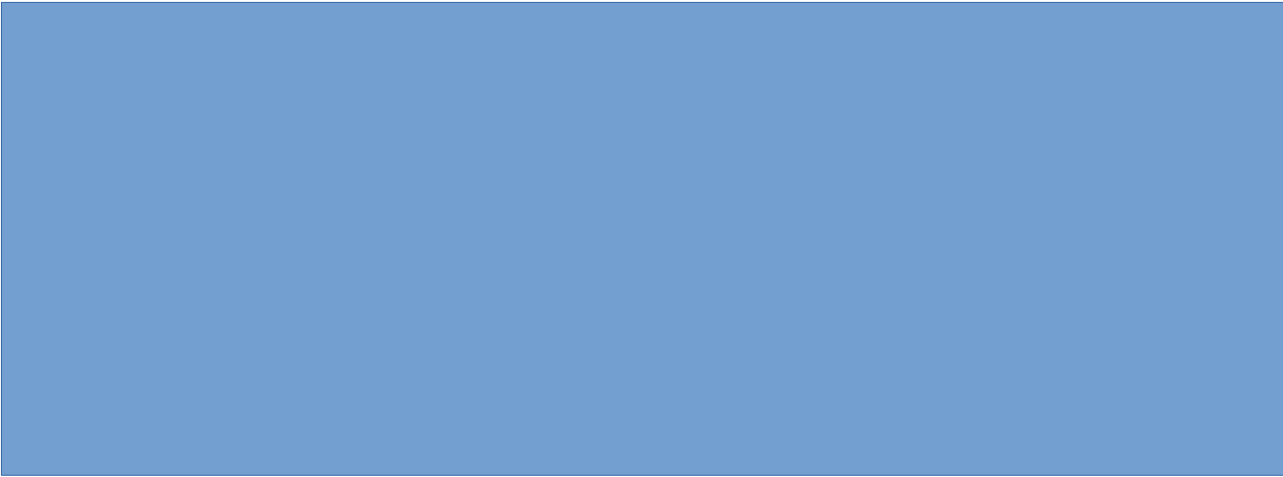
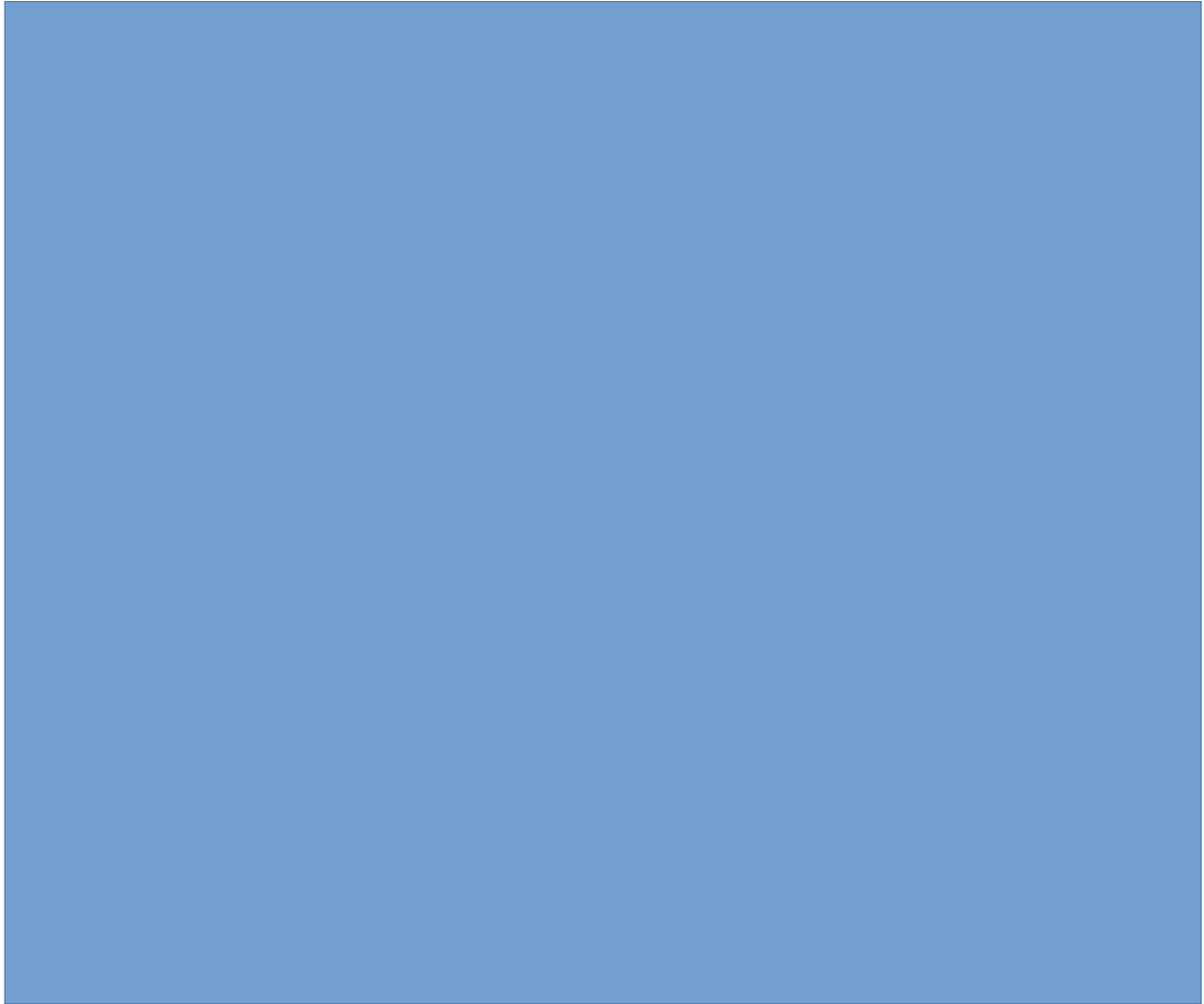




Clasificación de frutas con redes neuronales



Índice

Introducción.....	3
Análisis del dataset.....	3
Selección y entrenamiento del modelo.....	4
Arquitectura del modelo.....	5
Mejoras y especificaciones en la arquitectura del modelo.....	5
Evaluación del modelo y análisis de resultados.....	6
Producción en Streamlit.....	9
Producción con servidor Flask.....	10
Mejoras a futuro.....	11

Introducción

En este documento se detallará los procesos y soluciones propuestas para crear un clasificador de frutas usando una red neuronal. La descripción del problema es el siguiente:

Se debe automatizar la clasificación de 5 frutas distintas a través de fotos de ellas. Se debe clasificar correctamente entre **plátanos, manzanas, limones, naranjas y peras**. La solución debe poder ser accesible y utilizable tanto para usuarios finales como para el desarrollo de software.

Por ello, la clasificación será realizada por una CNN, la cual será entrenada desde cero con un dataset creado específicamente para este proyecto (dataset creado “a mano”, no obtenido de fuentes externas). Una vez se obtenga un modelo que cumpla con los requisitos y expectativas, se deberá:

- Poder montar un servidor API con el que realizar consultas para clasificar fotos de frutas.
- Una aplicación web que permita a los usuarios subir sus fotos de frutas y clasificarlas.

Análisis del dataset

El dataset utilizado son un conjunto de 286 imágenes donde 51 son de manzanas, 58 de plátanos, 62 de limones, 58 naranjas y 57 peras.

Las imágenes están almacenadas en forma de archivos jpg, con una resolución de 4032x3024 píxeles.

Dado que se utilizará la librería *dataset* de Hugging face para la gestión de este durante el entrenamiento de la CNN, la estructura de carpeta es la siguiente:

```
Fruits/train-----Apple
                        |---Banana
                        |---Lemon
                        |---Orange
                        |---Pear
```

El dataset presenta dos posibles peligros o detalles a tener en cuenta:

- De cara a la división de grupos (entrenamiento, validación y evaluación), la información de un mismo individuo no debe aparecer repetida en dos o más grupos, ya que se consideraría contaminación. Aplicado a las imágenes, las fotos de un mismo individuo no deberían aparecer en dos o más grupos distintos, sin embargo, dado que solo se busca un clasificador “sencillo” o experimental sin que tenga que ser extremadamente preciso o efectivo, se ha considerado que es mejor ignorar esta condición ya que requiere esfuerzos y precauciones mayores y más minuciosas que no llegan a compensar para lo que realmente se pretende como objetivo final.

- Entre las imágenes de manzanas, se encuentran mezcladas tanto manzanas rojas como verdes. Buscando que el clasificador, pueda clasificar ambos como manzanas.

Algunos ejemplos:



Selección y entrenamiento del modelo

Como se pretende hacer inferencia con imágenes donde el modelo debe detectar y clasificar la fruta que aparece en ella, se ha escogido la arquitectura de las CNNs como base para obtener y entrenar un modelo. Ya que estas, en vez de aprender de las imágenes pixel a pixel como lo haría una ANN, generan un mapa de características que permite a la red neuronal aprender contornos, formas, colores, texturas... Lo que permite una correcta clasificación de imágenes incluso aunque estas se “salieran del estándar”.

Dependiendo del contexto, podemos utilizar las CNNs para clasificar imágenes en escala de grises o a color, ya sea usando sistemas como RGB, RGBA... Es un error pensar que para entrenar una CNN lo mejor es usar imágenes con la mayor cantidad de información posible, por ejemplo, que usen sistema RGBA con una altísima resolución, y esperar que el sistema clasifique mejor que nada. Un exceso de información desvía la atención en el aprendizaje en detalles que pueden no ser tan importantes como otros, además de aumentar considerablemente los tiempos de entrenamiento, inferencia y complejidad del modelo. Por ello, es importante reducir la información a lo mínimo que realmente necesita el modelo para clasificar bien. Como nuestro caso es la clasificación de frutas, debemos pensar en que se caracteriza cada fruta, principalmente: en sus formas y colores. Por tanto, se requiere que nuestra CNN pueda reconocer esas formas y colores, pudiendo usar imágenes RGB en baja calidad.

Al principio, se consideró adaptar las imágenes a 512x512 píxeles, con la idea de que pueda también observar aprender de las texturas o patrones en la piel que las frutas suelen tener. Sin embargo, con los recursos hardware disponibles, los tiempos de entrenamiento eran demasiados extensos. Por lo que se ha reducido la escala a 64x64 píxeles, permitiendo estudiar formas y colores en tiempos razonables.



Arquitectura del modelo

Como ya se ha comentado y hablando en términos de la librería tensorflow. La arquitectura seleccionada será una **CNN**, por lo que se utilizarán capas **Conv2D** para la obtención del mapa de características de una imagen, luego por cada capa Conv2D se hará uso de una capa **MaxPooling2D**, reduciendo el tamaño del mapa para solamente procesar los datos de mayor importancia. A partir de ahora se denominará esta combinación como “grupo convolucional”. Después de aplicar grupos convolucionales, el mapa de características final será pasado por una o varias capas densas (**Dense**) que la procesarán y acabarán dando la clasificación final. La función de activación aplicable, tanto para las convencionales como para las densas, será ReLU, al ser una opción ampliamente estudiada y conocida al dar buenos resultados, incluso en el campo de las convolucionales. Dependiendo de las necesidades del modelo, se podrán aplicar uno o varios grupos convolucionales y capas densas. El modelo dará el resultado en formato One-Hot Encoding a través de una capa densa final usando la función de activación softmax.

Mejoras y especificaciones en la arquitectura del modelo

Uno de los mayores problemas de las redes neuronales es a su tendencia al sobreajuste y las CNNs no son la excepción. Por ello, se han aplicado algunas técnicas adicionales para controlar el sobreajuste durante los entrenamientos:

- **Dropouts:** Algunas neuronas de la red son “desactivadas” intencionalmente durante los entrenamientos, forzando a la red a recalcular sus pesos. Esta técnica no es solo aplicable a las capas **Dense**, también es aplicable a las capas convolucionales a través de **SpatialDropout2D**.
- **Early Stopping:** Llega un momento en los entrenamientos en los que la red se “estabiliza” y aprende todo lo que podía o debía aprender, pero si la red no es parada y siguen ejecutándose las épocas, la red intenta aprender los datos de entrenamiento al milímetro, llevándolo al sobreajuste, antes de que eso ocurra se aplica un **Early Stopping** para detener el entrenamiento antes de que ocurra.

Con respecto a la elección de propiedades específicas como los hiperparámetros, una de las cosas a tener en cuenta es que al modelo le entraran imágenes RGB de 64x64 píxeles, lo que equivale a la entrada de 12288 parámetros, por lo que la red neuronal va a requerir de unas capacidades mayores a las que tendría una red neuronal que se entrena con el dataset MNIST o Fashion (dataset de imágenes en escala de grises y de 28x28px, equivalente a 784 parámetros). Teniendo eso en mente, y tras varias pruebas se ha llegado a la siguiente arquitectura:

```
upgraded_model = tf.keras.Sequential([
    layers.Input(shape=(IMAGE_SIZE, IMAGE_SIZE, 3)),

    # Tensorflow data augmentation
    layers.RandomFlip("horizontal_and_vertical"),
    layers.RandomRotation(factor=0.5, fill_mode="reflect"),

    # Convolutional layers
    layers.Conv2D(filters=20, kernel_size=(7, 7), activation="relu"),
    layers.SpatialDropout2D(rate=0.15),
    layers.MaxPooling2D(pool_size=(4, 4)),

    layers.Conv2D(filters=15, kernel_size=(10, 10), activation="relu"),
    layers.SpatialDropout2D(rate=0.15),
    layers.MaxPooling2D(pool_size=(4, 4)),

    layers.Flatten(),

    # Dense layers
    layers.Dense(units=40, activation="relu"),
    layers.Dropout(rate=0.15),
    layers.Dense(units=40, activation="relu"),
    layers.Dropout(rate=0.15),

    layers.Dense(units=len(target_ids), activation="softmax")
])
```

Como vemos, se ha requerido dos capas densas y dos grupos convolucionales para que el modelo pudiera aprender. Dado que las imágenes son de 64x64px, se requiere que los grupos convolucionales tenga una mayor “capacidad de observación”, lo cual se logra agregando filtros, aumentando los tamaños de kernels, pero como eso contribuye al exceso de información, se ajustan también los tamaños de piscinas.

Evaluación del modelo y análisis de resultados

Para la evaluación de modelo se utilizarán las métricas **Accuracy**, **F1**, **Precision**, **Recall** y una **matriz de confusión**. Estas métricas suelen ser suficiente para analizar en profundidad a los modelos, no solo para ver que “clasifican bien”.

Los resultados del modelo final obtenidos con los siguientes conjunto de datos son:

- Entrenamiento

```
{'accuracy': 0.8571428571428571,  
 'f1': {'Apple': np.float64(0.7333333333333333),  
        'Banana': np.float64(0.7605633802816901),  
        'Lemon': np.float64(0.9230769230769231),  
        'Orange': np.float64(0.972972972972973),  
        'Pear': np.float64(0.8529411764705882)},  
 'precision': {'Apple': np.float64(0.8461538461538461),  
               'Banana': np.float64(0.75),  
               'Lemon': np.float64(0.875),  
               'Orange': np.float64(0.972972972972973),  
               'Pear': np.float64(0.8285714285714286)},  
 'recall': {'Apple': np.float64(0.6470588235294118),  
            'Banana': np.float64(0.7714285714285715),  
            'Lemon': np.float64(0.9767441860465116),  
            'Orange': np.float64(0.972972972972973),  
            'Pear': np.float64(0.8787878787878788)}}
```

- Validación

```
{'accuracy': 0.8478260869565217,  
 'f1': {'Apple': np.float64(0.8571428571428571),  
        'Banana': np.float64(0.75),  
        'Lemon': np.float64(0.8421052631578947),  
        'Orange': np.float64(1.0),  
        'Pear': np.float64(0.8)},  
 'precision': {'Apple': np.float64(0.8571428571428571),  
               'Banana': np.float64(0.75),  
               'Lemon': np.float64(0.8),  
               'Orange': np.float64(1.0),  
               'Pear': np.float64(0.8571428571428571)},  
 'recall': {'Apple': np.float64(0.8571428571428571),  
            'Banana': np.float64(0.75),  
            'Lemon': np.float64(0.8888888888888888),  
            'Orange': np.float64(1.0),  
            'Pear': np.float64(0.75)}}
```

- Evaluación

```
{'accuracy': 0.8275862068965517,  
  'f1': {'Apple': np.float64(0.6),  
        'Banana': np.float64(0.782608695652174),  
        'Lemon': np.float64(1.0),  
        'Orange': np.float64(1.0),  
        'Pear': np.float64(0.7741935483870968)},  
  'precision': {'Apple': np.float64(0.6),  
               'Banana': np.float64(0.75),  
               'Lemon': np.float64(1.0),  
               'Orange': np.float64(1.0),  
               'Pear': np.float64(0.8)},  
  'recall': {'Apple': np.float64(0.6),  
            'Banana': np.float64(0.8181818181818182),  
            'Lemon': np.float64(1.0),  
            'Orange': np.float64(1.0),  
            'Pear': np.float64(0.75)}}
```

A través del accuracy, podemos tener una visión general del modelo, donde podríamos decir que el modelo acierta un 80% de las veces. Sin embargo, si nos fijamos en las demás métricas, nos podemos fijar en que tiene algunos peros.

El modelo distingue bastante bien entre limones y naranjas. Le siguen las peras y los plátanos, donde falla un poco, pero en márgenes aceptables. Por último, y el más interesante, la clasificación de manzanas es más inestable: hay diferencias considerables entre los valores de la métrica f1 con las manzanas entre los tres conjuntos de datos. Lo más probable es que esto suceda porque dentro del conjunto de manzanas, hay imágenes tanto de manzanas verdes como rojas, y nuestro modelo distingue colores. Por tanto, podría ser que no haya aprendido ejemplos suficientes de ambos tipos de manzanas para que entienda que ambos son válidos. Existe la posibilidad de que el modelo se haya entrenado y validado principalmente con un tipo de manzana y que en el conjunto de evaluación hayan caído principalmente manzanas del tipo contrario al que se entrenó, lo que explicaría la diferencia abismal entre las métricas de cada conjunto.

Producción en Streamlit

Puedes crear una plataforma streamlit en español usando los recursos en la carpeta streamlit_app, desde la cual puedes probar el modelo:

Clasificador de Frutas con TensorFlow

Elige una imagen...

 2026040...7_noMD.jpg 0.8MB +



Imagen cargada

Clasificar

Predicción: **Naranja**

Confianza: 44.95%

▼ Ver salida cruda del modelo

0	1	2	3	4
0.1375	0.0677	0.2878	0.4495	0.0575

Producción con servidor Flask

También, se ha creado un servidor Flask junto con una aplicación cliente con Gradio para realizar peticiones.

Primero se debe instalar las dependencias con:

```
pip install -r flask_server/requirements.txt
```

Luego ejecutas la aplicación servidor:

- Linux

```
python3 -m flask_server.server
```

- Windows

```
python -m flask_server.server
```

Y, por ultimo, en una consola a parte, la aplicación cliente con Gradio:

- Linux

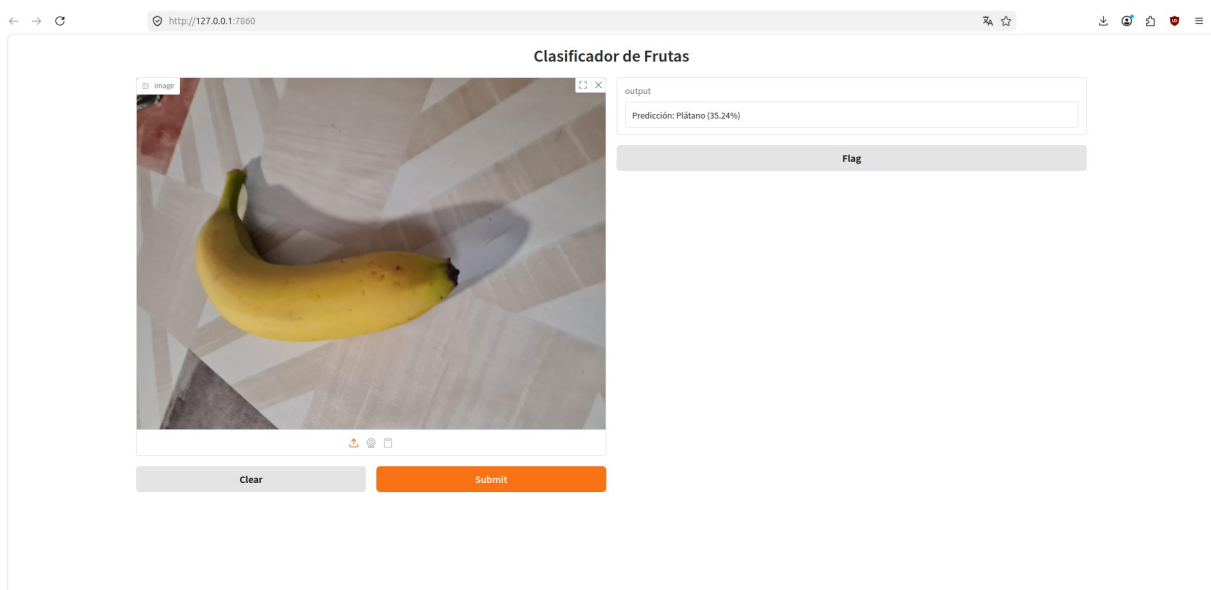
```
python3 -m flask_server.client
```

- Windows

```
python -m flask_server.client
```

Ahora, desde un navegador, buscas la URL que te indica la aplicación Gradio, por defecto

<http://127.0.0.1:7860>



Mejoras a futuro

Como primer objetivo, sería conveniente mejorar esos resultados con la clasificación de manzanas. Por lo que, aparte de simplemente aumentar la cantidad de imágenes de manzanas, también se podría buscar un dataset de mayor calidad, es decir, datos más “heterogéneos”. Ya que, como ya se ha explicado, un dataset de imágenes de los mismos individuos provocan que los datos sean más “homogéneos”, lo que también afecta a los entrenamientos y los resultados.

Como segundo objetivo, vemos con los ejemplos del servidor Flask y Streamlit, el modelo clasifica bien, pero lo hace “dudando”. En algunos casos los porcentajes de probabilidad en la predicción son bastante bajos. Por lo que, sería conveniente que el modelo lance clasificaciones donde esté seguro de lo que clasifica.